

I n f o r m a t i o n E x p e r t

Information Expert Principle

Give work to who has the data

DEFINITION

Assign a responsibility to the class that has the information needed to fulfil it.

Information Expert answers the most common OO question: which class should own a given method? The heuristic is simple — give the responsibility to the class that already holds the data it needs. "Expert" is not a role you design; it is whichever object happens to have the facts. Find the data, and the behaviour follows it there.

WHY IT MATTERS

Co-locating behaviour with its data is what makes encapsulation real: callers ask the owner for an answer instead of pulling out its fields and computing elsewhere ("tell, don't ask"). That keeps internals private, removes the same calculation duplicated across call sites, and lowers coupling — the data can change shape without breaking everyone who used to reach in.

— How it works

Expert	The class that already holds the data a responsibility needs — so it becomes the natural home for that behaviour.
Responsibility	The task being assigned: a calculation, decision, or query that operates on that data.
Information	The fields and collections the task needs. Trace them first; they point straight at the Expert.

HOW TO APPLY

1. Name the responsibility you need (e.g. "compute the order total").
2. List the data it needs to do its job.
3. Find the class that already owns most of that data — that is the Expert.
4. Put the method there; if it needs another object's data, let that object answer its own part.

LITMUS TEST

Who holds the data needed to answer this? If a method reaches into another object's fields to do its job, the responsibility is in the wrong place — move it to the data's owner.

— Smell → fix

A service reaches into Order's items to compute the total — Order's data, outside logic:

```
class Order { items: Item[] = [] }  
class Checkout {  
  total(o: Order) { // reaches into Order's data  
    return o.items.reduce(sum, 0) // outside logic  
  }  
}
```

↓ MOVE TO THE DATA OWNER

```
class Order {  
  items: Item[] = []  
  total() { // Order owns the data, so it computes  
    return this.items.reduce(sum, 0)  
  }  
}
```

The total now lives with the items it sums. Callers ask `order.total()` instead of fetching items and looping — state stays encapsulated.

USE WHEN

- ✓ Deciding which class gets a method
- ✓ Behaviour that needs a class's own data

AVOID WHEN

- ▲ When it bloats the data-holder or drags in I/O — use Pure Fabrication

— Trade-offs & pitfalls

PROS	CONS
+ Behaviour sits with its data + Encapsulation, low coupling	– Can overload one class – May clash with separation of concerns

COMMON PITFALLS

- ▲ Letting the Expert grow into a god class — when it pulls in unrelated duties, switch to Pure Fabrication.
- ▲ Forcing I/O or persistence onto a data-rich entity just because it "has the data" — that wrecks cohesion.
- ▲ The opposite mistake: anemic objects with data but no behaviour, so nothing is ever the Expert.

WHERE YOU SEE IT

- `order.total()` summing its own line items, instead of a service looping over them from outside.
- `cart.itemCount()` / `invoice.isOverdue()` — small queries living on the object that holds the state.
- A value object (`Money`, `DateRange`) doing its own arithmetic and comparisons.

SEE ALSO

High Cohesion	keeping behaviour near its data naturally raises cohesion
Low Coupling	callers stop reaching into internals — fewer, weaker dependencies
Pure Fabrication	the escape hatch when Expert would bloat a domain class with I/O
Creator	sibling heuristic — Expert places behaviour, Creator places construction

— Interview & sources

Information Expert vs SRP?

Expert decides where a responsibility goes (the data-holder); SRP limits how many reasons a class has to change. They can pull opposite ways — resolve the tension with Pure Fabrication.

Doesn't this create god classes?

It can. Expert is a default, not a mandate — when it would force unrelated duties (e.g. persistence) onto an entity, fabricate a Service or Repository instead.

How does it relate to "Tell, Don't Ask"?

Tell-Don't-Ask is Expert in practice: send the object a message instead of querying its data and acting on it from outside.

Is Information Expert always the right default?

It is the first heuristic, not the last word. When honouring it would push unrelated duties (I/O, cross-aggregate coordination) onto a data-holder, Pure Fabrication or a domain service overrides it to protect cohesion.

REMEMBER

Put the behaviour where the data already lives.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Information Expert

Martin Fowler — "TellDontAsk" (martinfowler.com)